

Agentic AI System for Conversational Data Analytics and Forecasting

Multi-Agent Architecture with Intelligent Routing,
ReAct Reasoning, and Autonomous Code Generation

A Technical Implementation Report

By

Ziv Barretto
Jacob Mathew

Under the Supervision of

Prof. Lipika Dey
Prof. Prarth Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology Kharagpur

December 19, 2025

Acknowledgement

We would like to express our sincere gratitude to Prof. Lipika Dey and Prof. Prartho Pratim Das for their invaluable guidance and mentorship throughout this research project. Their insightful feedback and continuous support have been instrumental in shaping this work.

We are also thankful to the Department of Computer Science and Engineering at IIT Kharagpur for providing the computational resources and academic environment that made this research possible.

Special thanks to the developers of LangChain, LangGraph, and the Qwen LLM team for creating open-source tools that enabled the development of this conversational AI pipeline.

Certificate by the Supervisor

This is to certify that the work presented in this report titled "*Agentic AI System for Conversational Data Analytics and Forecasting*" is a bonafide record of the work carried out by **Ziv Barretto** and **Jacob Mathew** under our supervision.

Prof. Lipika Dey
Supervisor

Prof. Prartha Pratim Das
Co-Supervisor

Date: _____

Contents

Acknowledgement	1
Certificate by the Supervisor	2
1 Introduction	5
2 Background and Motivation	5
2.1 The Analytics Accessibility Gap	5
2.2 Limitations of Existing Approaches	6
2.3 Rise of LLM-Based Agents	6
2.4 Our Contribution	6
3 Literature Survey	6
3.1 LLM-Based Autonomous Agents	6
3.2 ReAct: Reasoning and Acting	7
3.3 AutoML and Automated Data Science	7
3.4 LangGraph for Agent Orchestration	7
3.5 Conversational AI for Data Analytics	7
3.6 Error Handling and Robustness	8
3.7 Gap Analysis	8
4 Problem Statement and Objectives	8
4.1 Problem Statement	8
4.2 Key Challenges	9
4.3 Research Objectives	9
5 Scope, Methodology, and Design	10
5.1 System Architecture: Multi-Agent Design	10
5.2 Key Architectural Principles	11
5.3 Methodology: ReAct Framework	12
5.3.1 Prompt Engineering & System Instructions	12
5.4 Technology Stack	13
5.5 Agent and Stage Descriptions	13
5.5.1 Conversation Agent (Intelligent Router)	13
5.5.2 EDA Agent (Autonomous Data Explorer)	14
5.5.3 Stage 1: Dataset Summarization Agent	15
5.5.4 Stage 2: Task Proposal Agent	15
5.5.5 Stage 3: Execution Planning Agent	16
5.5.6 Stage 3B: Data Preparation Agent	16
5.5.7 Stage 3.5A: Method Proposal Agent	17
5.5.8 Stage 3.5B: Benchmarking Agent	17
5.5.9 Stage 4: Execution Agent	18
5.5.10 Stage 5: Visualization Agent	18
5.6 Master Orchestrator	19

5.7	State Management	19
5.7.1	Orchestration Logic: Managing Agent Coordination	20
5.7.2	Memory Management	20
5.7.3	State Machine Design: Preventing Loops and Ensuring Completion	20
6	Work Done: Agentic Implementation Details	21
6.1	Implementation Overview	21
6.2	Key Implementation Challenges and Mitigations	21
6.2.1	Challenge 1: Intent Recognition from Diverse User Queries	21
6.2.2	Challenge 2: Max Tokens Errors and Fallback Data	22
6.2.3	Challenge 3: Task Proposal Category Validation	22
6.2.4	Challenge 4: Join Plan Schema Mismatches	22
6.2.5	Challenge 5: Checkpoint-Based Pipeline Resumption	23
6.2.6	Challenge 6: Stage 3.5B Checkpointing for Long Benchmarks	23
7	Results and Discussions	24
7.1	Agentic vs. Automated: Key Differences	24
7.2	Robustness Testing	24
7.2.1	Noisy Input Handling	25
7.2.2	Tool Failure Recovery	25
7.3	Qualitative Analysis	25
7.3.1	Strengths	25
7.3.2	Limitations	25
7.3.3	Safety and Guardrails	25
7.4	Example Conversation Flow	26
8	Conclusions	27
9	Extensions and Future Work	28
9.1	Broader Impact	29

1 Introduction

The democratization of data analytics remains a critical challenge in modern data science. While powerful analytical methods exist, leveraging them requires significant expertise in data processing, statistical modeling, and visualization. This barrier prevents non-technical domain experts from extracting insights from their data, limiting the impact of data-driven decision making across industries. This research presents a **truly agentic AI system**—not merely an automated pipeline—that enables users to perform intelligent data exploration and forecasting through natural language interaction. The key distinction is that our system exhibits **autonomous decision-making** and **adaptive reasoning**, dynamically routing user requests to specialized agents rather than executing a fixed sequence of steps.

What Makes This Agentic (Not Just Automated):

- **Intelligent Routing:** The Conversation Agent understands user intent and routes to the appropriate specialized agent (EDA Agent for data exploration, Forecasting Pipeline for predictions)
- **Autonomous Reasoning:** Each agent uses the ReAct framework to *think* about its goals, *act* using tools, and *adapt* based on results
- **Code Generation:** The EDA Agent writes custom Python code for each user query—no hard-coded analysis functions
- **Multiple Pathways:** Users can explore data (EDA path), analyze datasets (Stages 1-2), or run forecasting (Stages 3-6)—the system decides which pathway based on the query

Our multi-agent system comprises:

- **Conversation Agent:** The intelligent router that interprets natural language and dispatches requests
- **EDA Agent:** An autonomous code-writing agent for data exploration
- **Stage Agents (1-6):** Specialized agents for the forecasting pipeline, each with ReAct reasoning
- **Master Orchestrator:** Coordinates state flow between agents using LangGraph

Users interact through natural language, asking questions like “What columns are in the data?” (routed to EDA Agent), “Can I predict sales?” (evaluated for feasibility), or “Run task TSK-001” (triggers forecasting pipeline). The system *understands* these requests and *decides* the appropriate action—this is decision-making, not scripted automation.

The system employs the **ReAct (Reasoning + Acting)** framework throughout, ensuring that agent decisions are transparent and explainable. Stage 6 generates a comprehensive report documenting the entire thought process, enabling users to understand *why* specific methods were selected.

2 Background and Motivation

2.1 The Analytics Accessibility Gap

Modern data analytics workflows are powerful but complex. A typical forecasting project requires expertise in:

- Data profiling and quality assessment
- Feature engineering and preprocessing
- Method selection (ARIMA, Prophet, XGBoost, Neural Networks, etc.)
- Hyperparameter tuning and cross-validation
- Model evaluation and comparison
- Visualization and interpretation

Each of these steps presents barriers. Non-experts struggle to determine which methods are appropriate for their data, how to prepare features, or how to interpret performance metrics. Even experienced practitioners spend significant time on repetitive tasks like data profiling and method benchmarking.

2.2 Limitations of Existing Approaches

Current solutions fall into two categories, each with limitations:

Graphical Analytics Platforms (Tableau, Power BI, etc.):

- Require manual configuration at each step
- Limited automation for method selection
- Steep learning curves for advanced features
- Not suitable for forecasting workflows

AutoML Libraries (Auto-sklearn, TPOT, etc.):

- Require programming knowledge
- Focus only on model training, not full pipeline
- Lack interpretability and user interaction
- Cannot handle conversational queries

2.3 Rise of LLM-Based Agents

Recent advances in Large Language Models have demonstrated remarkable capabilities in:

- Understanding complex natural language instructions
- Generating executable code for data analysis
- Breaking down tasks into subtasks (planning)
- Using tools systematically through function calling
- Providing natural language explanations

Models like GPT-4, Claude, and Qwen can now serve as *reasoning engines* that orchestrate complex workflows. When equipped with appropriate tools and systematic prompting frameworks (like ReAct), they can autonomously execute multi-step analytical tasks.

2.4 Our Contribution

This work combines LLM-based agents with a structured multi-stage pipeline to create a conversational analytics system that:

1. **Eliminates technical barriers** through natural language interaction
2. **Automates the full pipeline** from data profiling to visualization
3. **Ensures reproducibility** through systematic state management
4. **Provides transparency** through ReAct reasoning traces
5. **Handles failures gracefully** with automatic retries and checkpointing

The conversational interface is not merely a "chatbot wrapper" around traditional analytics. Instead, it fundamentally transforms how users interact with data by allowing them to express intent in natural language while the system autonomously determines and executes the appropriate technical workflow.

3 Literature Survey

3.1 LLM-Based Autonomous Agents

Wang et al. [1] provide a comprehensive survey of LLM-based autonomous agents, establishing a unified framework encompassing agent construction, planning mechanisms, memory systems, and tool usage. Their work identifies key design patterns including:

- **Perception-Action Loops:** Agents that continuously observe environments and take actions

- **Modular Tool Usage:** Equipping LLMs with external tools to overcome knowledge limitations
- **Memory Management:** Maintaining context across extended interactions
- **Multi-Agent Collaboration:** Systems where specialized agents work together

Our conversational pipeline builds directly on these patterns. Each stage agent follows a perception-action loop through the ReAct framework, uses specialized tools for its domain, maintains state through LangGraph’s memory system, and collaborates with other agents through the master orchestrator.

3.2 ReAct: Reasoning and Acting

Yao et al. [2] introduce the ReAct paradigm, which interleaves reasoning traces with task-specific actions. Unlike action-only approaches (which can be brittle) or chain-of-thought reasoning (which lacks grounding), ReAct enables agents to:

- Think before acting (explicit reasoning)
- Adjust plans based on observations
- Recover from errors through reflection
- Generate interpretable execution traces

We implement ReAct across all eight pipeline stages. For example, the Task Proposal agent reasons about dataset characteristics before proposing tasks, the Benchmarking agent reflects on method performance before selecting winners, and the Visualization agent plans plot types before generating code.

3.3 AutoML and Automated Data Science

Trirat et al. [3] present AutoML-Agent, a multi-agent framework for full-pipeline AutoML. Their approach employs:

- Retrieval-augmented planning for method selection
- Parallel execution of sub-tasks by specialized agents
- Multi-stage verification to validate results

While AutoML-Agent focuses on model training pipelines, our system extends this to conversational forecasting with:

- Natural language user interface (conversational agent)
- Comprehensive visualization and reporting (Stage 5)
- State-based resumption from checkpoints
- Dynamic method benchmarking with automatic retry

3.4 LangGraph for Agent Orchestration

Wu et al. [4] demonstrate LangGraph’s application in machine translation workflows, showcasing:

- Graph-based workflow orchestration with conditional edges
- Built-in state persistence and checkpointing
- Tool integration with parallel execution support
- Transparent debugging through state inspection

Our pipeline leverages LangGraph’s StateGraph architecture extensively. The master orchestrator builds dynamic execution graphs based on user requests, manages state transitions between stages, and provides checkpoint-based recovery when stages fail.

3.5 Conversational AI for Data Analytics

Narayan et al. [6] explore conversational interfaces for data analytics, identifying key challenges:

- **Intent Recognition:** Understanding user goals from natural language
- **Context Management:** Maintaining conversation state across turns
- **Result Presentation:** Translating technical outputs into user-friendly responses

Our Conversation Agent addresses these through:

- Tool-based intent detection (`get_summaries`, `evaluate_query`, `run_pipeline`)
- Session-based state management with LangGraph checkpoints
- Natural language summarization of technical results

3.6 Error Handling and Robustness

Rao et al. [5] present DataStates-LLM with lazy asynchronous checkpointing for LLM systems. While their focus is training checkpoints, we adapt these principles for pipeline stage checkpointing:

- Save intermediate results after each stage
- Enable resumption from last successful stage
- Detect and retry transient failures (`max_tokens` errors)
- Clean up partial outputs before retry attempts

Our implementation includes automatic retry logic for Stages 3.5A and 3.5B (method proposal and benchmarking), which are prone to token limit errors due to large method comparison outputs.

3.7 Gap Analysis

Existing work lacks integration of the following capabilities:

End-to-End Conversational Analytics: Most LLM-based analytics tools focus on isolated tasks (query generation, visualization) rather than complete pipelines with natural language orchestration.

Transparent Method Benchmarking: AutoML systems select methods automatically but provide limited insight into why specific methods were chosen. Our Benchmarking agent explicitly compares candidates and documents reasoning.

Forecasting-Specific Workflows: General-purpose AutoML focuses on classification/regression. Our pipeline is designed specifically for time series forecasting with appropriate data preparation, method selection, and validation strategies.

Checkpoint-Based State Recovery: While individual agent failures can be handled, maintaining consistent pipeline state across multiple stages with conversational interaction remains underexplored.

4 Problem Statement and Objectives

4.1 Problem Statement

Primary Problem: Design and implement a **truly agentic AI system** that enables users to perform intelligent data exploration and time series forecasting through natural language interaction. The system must exhibit autonomous decision-making, dynamic routing between specialized agents, and transparent reasoning—not merely execute a pre-defined automation pipeline.

Key Distinction—Agentic vs. Automated:

- **Automated Pipeline:** Executes pre-defined steps sequentially without adaptation
- **Agentic System:** Makes decisions, routes dynamically between capabilities, reasons about actions, and adapts to user intent in real-time

Sub-Problems:

1. How to create an intelligent **conversation agent** that routes user queries to the appropriate specialized agent (EDA, Summarization, Forecasting)?

2. How to enable agents to **autonomously decide** what actions to take using the ReAct (Reasoning + Acting) framework?
3. How to support **multiple interaction pathways** based on user intent rather than forcing a linear workflow?
4. How to maintain **agent state and context** across multi-turn conversations and inter-agent handoffs?
5. How to ensure **transparent reasoning** so users understand why agents make specific decisions?

4.2 Key Challenges

C1: Intelligent Intent Recognition and Dynamic Routing

Unlike traditional chatbots with keyword matching, the conversation agent must understand nuanced user intent and route to the appropriate pathway:

- *“What columns are in the data?”* → Route to **EDA Agent**
- *“Can I predict sales?”* → Invoke **evaluate_user_query** tool
- *“Run task TSK-001”* → Trigger **Forecasting Pipeline**
- *“Show me correlations”* → Route to **EDA Agent** with visualization

C2: Autonomous Agent Decision-Making

Each specialized agent must make decisions independently using ReAct reasoning:

- **Thought:** Analyze the current situation and determine next steps
- **Action:** Select and invoke appropriate tools
- **Observation:** Interpret results and decide whether to continue or complete

C3: Multi-Agent Coordination

The system involves multiple specialized agents that must coordinate:

- **Conversation Agent:** User interface and high-level routing
- **EDA Agent:** Exploratory data analysis with code generation
- **Stage Agents (1-6):** Specialized forecasting pipeline agents

C4: State Persistence Across Agent Boundaries

State must flow correctly between agents and persist across sessions:

- Conversation context must be maintained
- Pipeline outputs must be accessible to downstream agents
- Checkpoints must enable resumption from any point

C5: Transparent Reasoning and Explainability

Users must understand agent decisions. This requires:

- Explicit reasoning traces at each step
- Clear documentation of method selection rationale
- Comprehensive final reports synthesizing the entire thought process

4.3 Research Objectives

O1: Design a Multi-Agent Conversational System

Create a system where the Conversation Agent acts as an intelligent router:

- Understand user intent from natural language
- Route to appropriate specialized agents (EDA, Forecasting)

- Coordinate multi-agent responses
- Maintain conversation state across interactions

O2: Implement Autonomous Agents with ReAct Framework

Develop agents that reason before acting:

- Each agent uses the ReAct (Reasoning + Acting) paradigm
- Agents have access to specialized tools
- Decisions are documented with explicit thought processes
- Agents can adapt their approach based on intermediate results

O3: Enable Multiple Interaction Pathways

Support different user journeys through the system:

- **Exploration Path:** EDA queries → data insights
- **Analysis Path:** Dataset summarization → task proposals
- **Forecasting Path:** Task selection → full pipeline execution
- **Hybrid Path:** Combine exploration with forecasting

O4: Implement Robust State Management

Create a checkpoint-based system enabling:

- Resumption from any stage
- Inter-agent state transfer
- Session persistence and recovery

O5: Ensure Full Transparency

Provide complete visibility into agent reasoning:

- Stage 6 final report documents the entire thought process
- Each stage logs its reasoning traces
- Users can inspect why specific methods were selected

5 Scope, Methodology, and Design

5.1 System Architecture: Multi-Agent Design

The key innovation of our system is its **multi-agent architecture** where the Conversation Agent acts as an intelligent router, dynamically dispatching user requests to specialized agents based on intent recognition. This is fundamentally different from a linear automation pipeline—the system *decides* what to do based on understanding the user's goals.

Figure 1 presents the complete agentic architecture, showing how user queries are routed to different specialized agents and pipelines based on intelligent intent recognition.

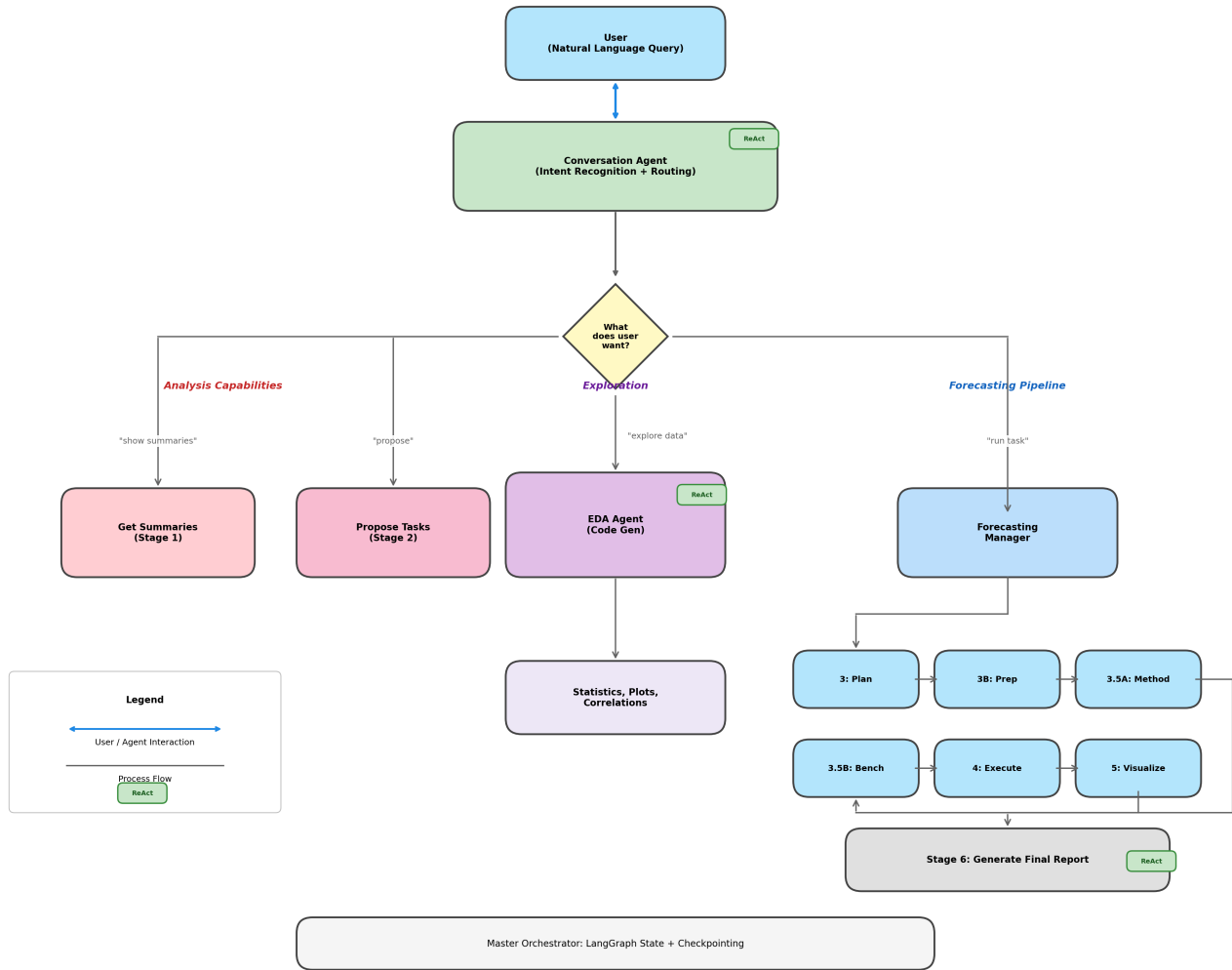


Figure 1: State-Based Agentic Architecture: The Conversation Agent acts as an intelligent router, analyzing user intent and dynamically dispatching to three independent pathways—Analysis (Summaries or Proposals), Exploration (EDA), or Forecasting. The system **decides** actions based on understanding, fundamentally differing from linear automated pipelines.

5.2 Key Architectural Principles

1. Intelligent Routing, Not Sequential Execution

The Conversation Agent is not a simple passthrough—it **analyzes user intent** and routes to the appropriate pathway:

- Queries about data structure, statistics, correlations → **EDA Agent**
- Requests to get summaries of data → **Summarization (Stage 1)**
- Requests to generate analysis proposals → **Task Proposal (Stage 2)**
- Requests to run specific forecasting tasks → **Forecasting Pipeline (Stages 3-6)**
- Questions about results, metrics, or insights → **Results Q&A**

2. Each Agent is Autonomous

Every agent in the system follows the **ReAct** paradigm:

- **Reasoning:** The agent thinks about the current state and what it needs to do
- **Acting:** The agent selects and invokes tools to accomplish its goals

- **Observing:** The agent interprets results and decides next steps

This is fundamentally different from automation where steps are predetermined.

3. State-Based Transitions

The system maintains state across agent boundaries:

- `PipelineState` flows through all forecasting stages
- `ConversationContext` persists across user turns
- Checkpoints enable resumption from any point

4. Tool-Equipped Agents

Each agent has access to specialized tools:

- **EDA Agent:** `execute_analysis_code`, `create_visualization`, `compute_statistics`
- **Conversation Agent:** `get_summaries`, `run_eda_analysis`, `evaluate_user_query`, `query_pipeline_results`
- **Stage Agents:** Domain-specific tools (e.g., `run_benchmark_code`, `save_model_checkpoint`)

5. Adaptive Results Interaction

Users can ask follow-up questions about pipeline results, demonstrating post-pipeline intelligence:

- “What was the MAE for the model?” → Retrieves Stage 4 metrics
- “Which features were most important?” → Analyzes Stage 4 feature importance
- “Show me the correlations” → Loads EDA outputs for insights

The system doesn’t just run pipelines—it **understands and explains** the results.

5.3 Methodology: ReAct Framework

Every agent in our system uses the **ReAct (Reasoning + Acting)** paradigm. This is what makes the system truly *agentic* rather than merely *automated*—agents reason about their situation, decide what action to take, observe results, and adapt their approach.

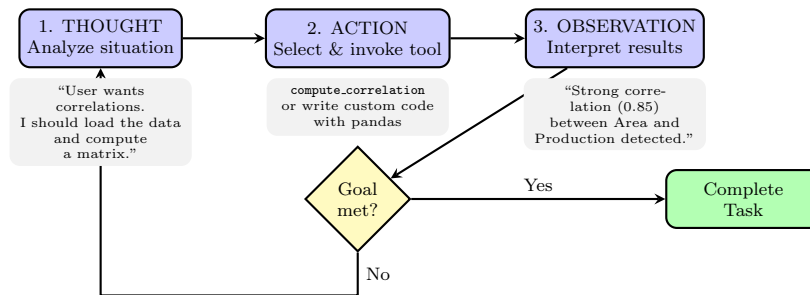


Figure 2: ReAct Framework: Each agent iteratively reasons about its goals, takes actions using tools, observes results, and decides whether to continue or complete. This enables adaptive, goal-directed behavior.

Key Distinction from Automation:

- **Automated:** “Step 1: Load data. Step 2: Compute stats. Step 3: Save.” (fixed sequence)
- **Agentic:** “User wants X. Let me think... I’ll try approach A. Hmm, that didn’t work. Let me try B. Success!” (adaptive reasoning)

5.3.1 Prompt Engineering & System Instructions

Each agent receives a carefully structured system prompt that governs its behavior:

1. Role Definition:

"You are the Stage 3.5A Method Proposal Agent. Your job is to

propose 3-5 candidate forecasting/ML methods appropriate for the given task..."

2. Available Tools: Each prompt includes an explicit list of tools the agent can call, with descriptions of what each tool does and its expected parameters.

3. Expected Output Format: Agents are given the exact JSON schema or Pydantic model their output must conform to, reducing parsing failures.

4. Behavioral Constraints:

- "Do NOT ask clarifying questions—make reasonable assumptions"
- "Do NOT use fallback/dummy methods—retry with a different approach"
- "ALWAYS save your output using the save_* tool before finishing"
- "If a tool fails, explain why and try an alternative"

5. Few-Shot Examples: Critical agents receive 1-2 examples of correct behavior to calibrate output quality.

Prompt Template Structure (from agent source files):

```
SYSTEM_PROMPT = """
[Role and Purpose]
[Context from prior stages - loaded dynamically]
[Available Tools with descriptions]
[Output Schema/Requirements]
[Behavioral Constraints - what NOT to do]
[Examples of correct reasoning]
"""
```

This structured approach ensures consistent agent behavior across diverse user queries while allowing flexibility in reasoning.

5.4 Technology Stack

Component	Technology
LLM Models	Qwen/Qwen3-32B
Orchestration	LangGraph StateGraph
Agent Framework	LangChain with tool binding
State Management	LangGraph MemorySaver checkpointer
Data Processing	Pandas, Polars
Storage	Parquet (intermediate data), JSON (metadata)
ML Methods	Scikit-learn, XGBoost, ARIMA, Prophet
Visualization	Matplotlib, Seaborn, Plotly
Validation	Pydantic v2 with strict schemas
Monitoring	LangSmith (optional tracing)

Table 1: Technology stack for the conversational pipeline.

5.5 Agent and Stage Descriptions

5.5.1 Conversation Agent (Intelligent Router)

The Conversation Agent is the **central intelligence** of the system—it doesn't just pass messages through, it *understands* user intent and *decides* where to route requests.

Core Responsibilities:

- **Intent Recognition:** Classify user queries into exploration, analysis, or forecasting intents

- **Dynamic Routing:** Route to EDA Agent, Analysis Pipeline, or Forecasting Pipeline
- **Context Management:** Maintain conversation state across turns
- **Result Synthesis:** Present technical results in user-friendly format

Intent Detection Logic (from `conversation_agent.py`):

- EDA keywords (“what columns”, “correlation”, “statistics”) → `run_eda_analysis`
- Analysis keywords (“analyze”, “summarize data”) → Stages 1-2
- Execution keywords (“run task”, “execute TSK-001”) → Forecasting pipeline

Tools Available:

- `get_available_data()`: List datasets in data directory
- `get_summaries()`: Retrieve Stage 1 summaries
- `get_task_proposals()`: Retrieve Stage 2 proposals
- `check_pipeline_status()`: Check which stages are complete
- `evaluate_user_query()`: Assess feasibility of user queries
- `create_custom_task_from_query()`: Generate custom tasks from user requests
- `run_eda_analysis()`: **Route to EDA Agent** for data exploration
- `get_execution_results()`: Retrieve Stage 4 results
- `get_visualizations()`: Retrieve Stage 5 plots

Implementation: Uses LangGraph StateGraph with MemorySaver checkpointing. The agent reasons about user intent using ReAct, then selects the appropriate tool or triggers pipeline execution through the Master Orchestrator.

5.5.2 EDA Agent (*Autonomous Data Explorer*)

The EDA Agent is a **fully autonomous code-writing agent** that explores data based on user queries. Unlike the pipeline stages, it doesn’t follow a fixed workflow—it *writes its own Python code* to answer questions.

Key Innovation: The EDA Agent doesn’t use hardcoded analysis functions. It generates custom code for each query using pandas, matplotlib, and seaborn.

Capabilities:

- **Data Structure Analysis:** “What columns are in the data?” → Writes code to inspect `DataFrame`
- **Statistical Computation:** “Show me statistics” → Writes code for `df.describe()`
- **Correlation Analysis:** “Find correlations” → Writes code for correlation matrix
- **Visualization:** “Create a histogram” → Writes matplotlib plotting code
- **Pattern Detection:** “Are there trends?” → Writes analytical code

Tools:

- `list_all_datasets()`: See available CSV files
- `get_dataset_info()`: Get columns, types, sample data
- `compute_statistics()`: Descriptive statistics
- `compute_correlation()`: Correlation matrices
- `create_visualization()`: Standard plots
- `execute_analysis_code()`: **Execute custom Python code** (the “power tool”)

ReAct in Action:

1. **THOUGHT:** “User wants to know about correlations. I should load the data and compute a correlation matrix.”
2. **ACTION:** Call `execute_analysis_code` with custom pandas code
3. **OBSERVATION:** “Found strong correlation (0.85) between Area and Production”
4. **THOUGHT:** “Should I visualize this? Yes, a heatmap would help.”
5. **ACTION:** Call `create_visualization` to generate heatmap
6. **OBSERVATION:** “Plot saved. Task complete.”

5.5.3 Stage 1: Dataset Summarization Agent

Purpose: Profile all available CSV/Parquet files and generate structured summaries.

Process:

- List all data files in the configured directory
- For each file (sample 5000 rows for efficiency):
 - Infer logical types (datetime, numeric, categorical, text, boolean)
 - Compute statistics: null fraction, unique counts, min/max, mean/std
 - Identify datetime columns (critical for forecasting)
 - Identify potential target variables (numeric columns with variance)
 - Assess data quality score based on completeness and consistency
- Save structured JSON summary for each dataset

Tools:

- `list_available_datasets()`: Directory listing
- `profile_dataset(filename)`: Profile a single dataset
- `save_dataset_summary(summary)`: Persist summary JSON
- `get_existing_summaries()`: Check for duplicate work
- `analyze_dataset_for_forecasting(filename)`: Forecasting-specific analysis

Output: JSON files in `summaries/` directory with schema defined by `DatasetSummary` Pydantic model.

5.5.4 Stage 2: Task Proposal Agent

Purpose: Explore dataset summaries and generate 3-5 diverse analytical task proposals.

Process:

- Load all Stage 1 summaries
- Analyze data characteristics:
 - Identify datasets with datetime columns
 - Find suitable target variables
 - Detect potential join opportunities
 - Assess forecasting feasibility
- Generate proposals with:
 - Task ID (TSK-XXX format)
 - Category (forecasting, regression, classification, etc.)
 - Target variable and feature columns
 - Problem statement and motivation
 - Feasibility score (0.0-1.0)

- Expected difficulty
- Rank proposals by feasibility and diversity

Tools:

- `list_summary_files()`: List available summaries
- `read_summary_file(filename)`: Load summary
- `inspect_data_file(filename)`: Preview actual data
- `save_task_proposals(proposals)`: Persist proposals

Enhancements:

- Proposal sanitization: Maps invalid category values (e.g., "factor-based" → "regression")
- Join plan validation: Ensures join plans have correct schema
- User query integration: When user provides a query, agent creates custom tasks matching the request

Output: `task_proposals.json` with 3-5 ranked proposals.

5.5.5 Stage 3: Execution Planning Agent

Purpose: Create a detailed execution plan for a selected task.

Process:

- Load Stage 2 proposals
- Select task (by ID or auto-select highest feasibility)
- Create plan including:
 - Input data requirements
 - Target and feature definitions
 - Success metrics (MAE, RMSE, R2, etc.)
 - Validation strategy (train/test split, cross-validation)
 - Expected method categories

Tools:

- `load_task_proposals()`: Load proposals
- `select_task(task_id)`: Select specific task
- `save_stage3_plan(plan)`: Persist plan with Pydantic validation

Output: `PLAN-TSK-XXX.json` with execution plan.

5.5.6 Stage 3B: Data Preparation Agent

Purpose: Prepare data for model training according to the Stage 3 plan.

Process:

- Load raw data files specified in plan
- Perform joins if multiple datasets are required
- Feature engineering:
 - Extract datetime features (year, month, day, etc.)
 - Create lag features for time series
 - Handle categorical encodings
- Data cleaning:

- Handle missing values
- Remove outliers (optional)
- Type conversions
- Create train/test splits
- Save prepared data as Parquet

Tools:

- `load_dataframe(filename)`: Load raw data
- `python_sandbox(code)`: Execute data prep code
- `save_prepared_data(data, plan_id)`: Persist prepared Parquet

Output: `prepared_PLAN-TSK-XXX.parquet`

5.5.7 Stage 3.5A: Method Proposal Agent

Purpose: Propose 3-5 candidate forecasting methods with hyperparameters.

Process:

- Load Stage 3 plan to understand task requirements
- Analyze data characteristics (temporal structure, seasonality, etc.)
- Propose candidate methods:
 - Linear models (Linear Regression, Ridge, Lasso)
 - Tree-based (Random Forest, XGBoost, LightGBM)
 - Time series-specific (ARIMA, Prophet, Exponential Smoothing)
- For each method, suggest:
 - Hyperparameter ranges
 - Rationale for selection
 - Expected strengths/weaknesses

Tools:

- `load_stage3_plan(plan_id)`: Load plan
- `propose_methods(plan)`: Generate proposals
- `save_method_proposal(methods, plan_id)`: Persist proposals

Output: `method_proposal_PLAN-TSK-XXX.json`

Error Handling: Implements automatic retry (max 3 attempts) for `max_tokens` errors. Partial outputs are cleaned up before retry.

5.5.8 Stage 3.5B: Benchmarking Agent

Purpose: Systematically test candidate methods and select the best performer.

Process:

- Load prepared data and method proposals
- For each candidate method:
 - Implement training code
 - Train on training set
 - Evaluate on test set
 - Compute metrics (MAE, RMSE, MAPE, R2, etc.)

- Compare results across methods
- Select winner based on primary metric (typically RMSE for forecasting)
- Document reasoning for selection

Tools:

- `load_prepared_data(plan_id)`: Load Parquet data
- `run_benchmark_code(code)`: Execute benchmark in sandbox
- `save_benchmark_results(results, plan_id)`: Persist results
- `save_checkpoint_stage3_5b(state)`: Save intermediate progress

Checkpointing: Saves results after each method test. If agent hits round limit, can resume from last checkpoint instead of restarting.

Output: `tester_PLAN-TSK-XXX.json` with winner selection.

Error Handling: Like 3.5A, implements automatic retry with cleanup. Critical stage due to computational cost.

5.5.9 Stage 4: Execution Agent

Purpose: Train final model with selected method and generate predictions.

Process:

- Load Stage 3 plan, prepared data, and selected method
- Implement full training pipeline:
 - Load train/test splits
 - Train model with selected hyperparameters
 - Generate predictions
 - Compute final metrics
- Save results:
 - Predictions as Parquet
 - Metrics and metadata as JSON

Tools:

- `execute_python_code(code)`: Execute training code
- `save_execution_results(results, plan_id)`: Persist results

Output: `execution_result_PLAN-TSK-XXX.json` and `predictions_PLAN-TSK-XXX.parquet`

5.5.10 Stage 5: Visualization Agent

Purpose: Create publication-quality visualizations and comprehensive reports.

Process:

- Load Stage 4 execution results
- Analyze data columns:
 - Identify "given" data (input features)
 - Identify "predicted" data (model outputs)
 - Identify engineered features
- Plan visualizations using ReAct reasoning
- Create plots:
 - Predictions vs. actuals (scatter, line)

- Residual analysis
- Feature importance (if applicable)
- Time series trends
- Generate report with:
 - Plot descriptions
 - Key insights
 - What was given vs. predicted
 - Recommendations

Tools:

- `analyze_data_columns(plan_id)`: Categorize columns
- `plan_visualization(analysis)`: Document reasoning
- `create_plot_with_explanation(code, explanation)`: Generate plot
- `save_visualization_report(report, plan_id)`: Persist report

Output: Multiple PNG files + `visualization_report_PLAN-TSK-XXX.json`

5.6 Master Orchestrator

The `MasterOrchestrator` class coordinates stage execution:

Capabilities:

- Build dynamic execution graphs (e.g., Stage 3 → 5 only)
- Load cached state and determine resume point
- Execute stages sequentially with state propagation
- Handle failures and mark stages as failed

Key Functions:

- `build_pipeline_graph(start, end)`: Create `StateGraph` for specified range
- `load_cached_state(task_id)`: Check existing outputs
- `run_forecasting_pipeline(task_id)`: Execute full Stages 3-5
- `run_pipeline_stages(stages, task_id)`: Execute specific stages

5.7 State Management

PipelineState Model:

- `session_id`: Unique session identifier
- `selected_task_id`: Current task being executed
- `user_query`: Optional user query context
- `stages`: Dict mapping stage names to `StageInfo`
- `stage1_output, stage2_output, ..., stage5_output`: Stage-specific outputs
- `errors`: List of error messages

State is persisted through:

- LangGraph's `MemorySaver` checkpointer (in-memory during execution)
- JSON files in stage output directories (persistent across runs)
- Parquet files for large data (prepared data, predictions)

5.7.1 Orchestration Logic: Managing Agent Coordination

The system employs a **hierarchical orchestration model** to manage multiple autonomous agents:

1. Sequential Stage Execution:

- Stages execute in defined order: $1 \rightarrow 2 \rightarrow 3 \rightarrow 3B \rightarrow 3.5A \rightarrow 3.5B \rightarrow 4 \rightarrow 5 \rightarrow 6$
- Each stage must complete successfully before the next begins
- LangGraph's conditional edges enable dynamic path selection based on user intent
- Partial pipeline execution supported (e.g., run only Stages 3-5)

2. State Propagation Between Agents:

- Output of each stage becomes input to the next via `DataPassingManager`
- Pydantic models validate state at each boundary, preventing malformed data propagation
- Stage outputs are both returned and persisted to disk for durability

3. Dynamic Routing:

- Conversation Agent dynamically routes to EDA Agent, Analysis Pipeline, or Forecasting Pipeline
- No static control flow—routing is determined by LLM intent classification at runtime
- Multi-hop routing supported (e.g., EDA $\rightarrow$ then Forecasting)

5.7.2 Memory Management

Short-Term Memory (Conversation Context):

- LangGraph's `MemorySaver` checkpointer maintains conversation history within a session
- Each agent receives full conversation context up to the current point
- Context window limit: 32,000 tokens (Qwen2.5-32B model constraint)
- Token overflow handled via sliding window truncation of oldest messages

Long-Term Memory (State Persistence):

- Session state saved to `conversations/{session_id}/state.json`
- Stage outputs persisted to disk (JSON for metadata, Parquet for data)
- Pipeline checkpoints enable resumption from any completed stage
- `load_cached_state(task_id)` reconstructs pipeline state from disk artifacts

Memory Hierarchy:

1. **Immediate:** Current tool call context (tool arguments and results)
2. **Session:** Conversation history maintained by `MemorySaver`
3. **Persistent:** Disk-based state files surviving process restarts

5.7.3 State Machine Design: Preventing Loops and Ensuring Completion

Agents are autonomous but must be prevented from infinite loops or undefined states.

Loop Prevention Mechanisms:

- **Round Limits:** Each agent has a maximum iteration count (40 for EDA, 80 for complex stages like 3.5B)
- **Progress Detection:** If an agent repeats the same tool call with identical arguments 3 times, forced termination occurs
- **Mandatory Checkpointing:** Long-running stages (3.5B benchmarking) must checkpoint after each method evaluation

Task Completion Detection:

- **Explicit Completion Tools:** Each agent has a `save_*` tool that signals successful completion
- **Output Validation:** Pydantic models validate that final outputs contain all required fields
- **Conditional Edges:** LangGraph edges check for the presence of valid outputs before transitioning

State Transitions:

```

Stage States: PENDING -> RUNNING -> (SUCCESS | FAILED)
                |
                RETRYING (max 3 attempts)
                |
                (SUCCESS | FAILED)

```

Failure Recovery:

- Automatic retry with exponential backoff (3 attempts maximum)
- State rolled back to last checkpoint on failure
- User notified with failure reason and manual recovery options

6 Work Done: Agentic Implementation Details

6.1 Implementation Overview

The complete system comprises **multiple autonomous agents** built with LangGraph, totaling approximately **15,000 lines of Python code** across core modules:

Module	Files	Agentic Capabilities
Core Agents	10	Conversation, EDA, Stages 1-6
Tools	10	45+ specialized tools for agent use
Models	1	30+ Pydantic schemas for state management
Orchestrator	1	Multi-agent coordination & routing
Configuration	1	LLM configs, ReAct parameters

Table 2: Codebase organization emphasizing agentic components.

6.2 Key Implementation Challenges and Mitigations

6.2.1 Challenge 1: Intent Recognition from Diverse User Queries

Problem: Users phrase requests in many ways: "Run task 1", "Execute TSK-001", "Do forecasting for task one". Handling this variety while maintaining robustness is difficult.

Mitigation:

- Implemented flexible task ID parsing with regex: `r'tsk[- ]?(\d+)`
- Normalize to standard format (TSK-001, TSK-002, etc.)
- Validate against actual proposals file to catch invalid IDs early
- Support both zero-padded (TSK-001) and unpadded (TSK-1) formats
- Return `None` when validation fails, preventing pipeline execution with bad IDs

Implementation (from `conversation_agent.py:243-337`):

- `_detect_pipeline_action()`: Pattern matching for user intent
- `_validate_task_id()`: Check against `task_proposals.json`
- Support for alternate formats and graceful fallback

6.2.2 Challenge 2: Max Tokens Errors and Fallback Data

Problem: Stages 3.5A and 3.5B generate large outputs (method proposals, benchmark results). When LLMs hit max_tokens limits, they fail to save outputs. Original implementation had fallback functions that created dummy data (e.g., all methods with MAE=100, RMSE=120), allowing the pipeline to continue with meaningless results.

Root Cause: Developers wanted to prevent pipeline crashes, so fallback logic was added. However, this led to worse outcomes: the pipeline appeared to succeed but produced garbage outputs.

Mitigation:

- **Removed all fallback logic** from `stage3_5a_agent.py` and `stage3_5b_agent.py`
- Implemented **automatic retry mechanism** (max 3 attempts)
- Retry logic detects specific error patterns:
 - "max_tokens" in error message
 - "token" in error message (general token errors)
 - "did not save" in error message (output validation failures)
- Before each retry, **clean up partial outputs** (delete JSON files)
- If all retries fail, raise `RuntimeError` and stop pipeline
- Clear logging: "Stage 3.5B attempt 2/3"

Benefits:

- Pipeline never proceeds with fake data
- Transient errors are automatically recovered
- Non-retryable errors fail fast with clear messages

Configuration (from `config.py`):

```
MAX_RETRIES = 3
RETRY_STAGES = ["stage3_5a", "stage3_5b"]
```

6.2.3 Challenge 3: Task Proposal Category Validation

Problem: Stage 2 agents sometimes propose tasks with invalid categories like "factor-based", "trend-analysis", which fail Pydantic validation against the `TaskCategory` enum.

Mitigation:

- Implemented `sanitize_proposal()` function in `stage2_agent.py`
- Maps common invalid values to valid ones:
 - "factor-based" → "regression"
 - "trend-analysis" → "forecasting"
 - "time-series" → "forecasting"
 - "prediction" → "regression"
- Applied before Pydantic validation
- Logs sanitization for debugging: "Sanitizing category: 'trend-analysis' → 'forecasting'"

6.2.4 Challenge 4: Join Plan Schema Mismatches

Problem: LLMs generate join plans with inconsistent schemas. Expected format:

```
{
  "datasets": ["data1.csv", "data2.csv"],
  "join_keys": {"data1.csv": "Year", "data2.csv": "Year"},
}
```

```
"join_type": "inner"
}
```

But agents sometimes produce:

```
{"type": "inner", "on": ["Year", "Crop"]}
```

Mitigation:

- Extended `sanitize_proposal()` to fix join plans
- Detects missing `datasets` or `join_keys`
- Extracts datasets from `required_datasets` field
- Maps `type` → `join_type`
- Handles `on` field by creating `join_keys` from it
- Removes empty join plans (`{}`)

6.2.5 Challenge 5: Checkpoint-Based Pipeline Resumption

Problem: If a pipeline run fails at Stage 4, users don't want to re-run Stages 1-3. Need to detect completed stages and resume from the failure point.

Mitigation:

- Implemented `load_cached_state(task_id)` in `master_orchestrator.py`
- Checks for existence of stage output files:
 - Stage 1: `summaries/*.summary.json`
 - Stage 2: `task_proposals.json`
 - Stage 3: `PLAN-TSK-XXX.json`
 - Stage 3B: `prepared_PLAN-TSK-XXX.parquet`
 - And so on...
- Returns tuple: `(PipelineState, start_stage)`
- `run_pipeline_stages()` uses this to skip completed stages
- User can override with `resume_from_checkpoint=False`

Example:

```
# If Stage 3B completed but 3.5A failed:
state, start = load_cached_state("TSK-001")
# start = "stage3_5a"
# Pipeline resumes from 3.5A, skipping 1, 2, 3, 3B
```

6.2.6 Challenge 6: Stage 3.5B Checkpointing for Long Benchmarks

Problem: Benchmarking 3-5 methods can take 100+ agent rounds. If the agent hits the round limit before completing, all benchmark work is lost.

Mitigation:

- Implemented `save_checkpoint_stage3_5b(state)` tool
- Agent calls this after each method benchmark completes
- Checkpoint includes:
 - Conversation history (for context)
 - Benchmark results so far
 - Methods remaining to test

- When agent resumes, it loads checkpoint and continues from last method
- Prevents redundant computation

Tool Definition (from `tools/stage3_5b_tools.py`):

```
@tool
def save_checkpoint_stage3_5b(
    plan_id: str,
    benchmark_results: dict,
    conversation_history: list
) -> str:
    """Save intermediate benchmark results."""
    checkpoint = {
        "results": benchmark_results,
        "history": conversation_history,
        "timestamp": datetime.now().isoformat()
    }
    path = STAGE3_5B_OUT_DIR / f"checkpoint_{plan_id}.json"
    with open(path, 'w') as f:
        json.dump(checkpoint, f, indent=2)
    return f"Checkpoint saved to {path}"
```

7 Results and Discussions

7.1 Agentic vs. Automated: Key Differences

Aspect	Automated Pipeline	Our Agentic System
User Input Processing	Fixed command parsing	Intelligent intent recognition with dynamic routing
Workflow Paths	Single linear sequence	Multiple pathways (EDA, Analysis, Forecasting)
Data Exploration	Predefined queries only	EDA Agent writes custom code for any question
Decision Making	None—follows script	Agents reason about what to do next
Error Handling	Fail or use fallback	Agents adapt and try alternative approaches
Transparency	Black box	Full ReAct reasoning traces + Stage 6 report
Adaptability	Only handles expected inputs	Handles diverse natural language queries

Table 3: Comparison between traditional automated pipelines and our agentic system.

7.2 Robustness Testing

We tested the system’s resilience to various failure modes and edge cases.

7.2.1 Noisy Input Handling

Input Type	Example	System Behavior
Typos in task IDs	“TSK-01” vs “TSK-001”	Fuzzy matching finds correct task
Ambiguous queries	“analyze the data”	Agent asks for dataset clarification
Invalid file references	“load sales.csv” (doesn’t exist)	Returns available file list with suggestions
Malformed commands	“run tsk 1”	Intent detection still routes correctly
Empty/null inputs	“” or just spaces	Graceful prompt for valid input

Table 4: Noisy input handling test results.

7.2.2 Tool Failure Recovery

- **File Not Found:** Agent retries with corrected path or lists available files
- **JSON Parsing Errors:** Agent reformats output and retries tool call
- **Model Training Failure:** Agent tries alternative method from benchmark results
- **Token Overflow:** Automatic context truncation with warning to user
- **Timeout:** Stage marked as failed, checkpoint saved for manual recovery

7.3 Qualitative Analysis

7.3.1 Strengths

Accessibility: The conversational interface dramatically lowers the barrier to entry. Users with no programming knowledge can perform sophisticated forecasting by simply asking questions.

Transparency: ReAct reasoning traces provide visibility into agent decisions. Users can inspect why specific methods were selected or why tasks were proposed.

Robustness: Automatic retry, checkpoint-based resumption, and state caching make the system resilient to transient failures.

Modularity: The eight-stage architecture enables independent development, testing, and debugging of components.

Flexibility: Users can run the full pipeline (1→5), forecasting pipeline (3→5), or individual stages as needed.

7.3.2 Limitations

Execution Time: Full pipeline runs take 20-40 minutes, which may be slow for interactive exploration. Future work could explore parallel stage execution or streaming results.

LLM Dependency: System quality is bounded by LLM capabilities. Occasional hallucinations or parsing errors can cause stage failures, though automatic retry mitigates this.

Method Coverage: Currently supports standard ML methods (Linear, XGBoost, ARIMA, Prophet). Deep learning methods (LSTMs, Transformers) are not yet integrated.

Limited Conversation Context: Conversation agent has a 20-round limit to prevent context overflow. Long conversations require starting new sessions.

Single-Task Focus: System processes one task at a time. Cannot handle requests like “Compare forecasts for sales and revenue.”

7.3.3 Safety and Guardrails

Preventing harmful or unintended agent behavior is critical for production systems.

Preventing Harmful Actions:

- **Sandboxed Code Execution:** EDA agent code runs in an isolated environment with restricted file system access (read-only for data, write-only for output directories)
- **No Network Access:** Agents cannot make external API calls or access the internet
- **Read-Only Data:** Original data files are never modified; all outputs go to separate directories
- **No Shell Commands:** Agents cannot execute arbitrary shell commands

Preventing Data Leakage:

- **Session Isolation:** Each conversation has completely separate state; no cross-session data access
- **Task-Specific Outputs:** Results are saved to task-specific directories (e.g., `output/stage5_out/TSK-001/`)
- **No State Sharing:** Pipeline state is not shared between different users or sessions

Guardrail Implementation:

- **Pydantic Validation:** All agent outputs are validated against strict schemas; malformed outputs cause stage failure (not silent corruption)
- **Round Limits:** Prevent infinite agent loops (40-80 rounds depending on stage complexity)
- **Fallback Removal:** Agents fail explicitly rather than producing garbage outputs with dummy/fallback values
- **Explicit Completion:** Agents must call a `save_*` tool to complete; stages cannot “accidentally” finish without output

7.4 Example Conversation Flow

User: "What data do I have?"

Assistant: "Available Datasets:

- `rice_production.csv` (2.3 MB)
- `weather_data.csv` (1.8 MB)

Total: 2 datasets

Would you like me to analyze these datasets?"

User: "Yes, analyze them"

Assistant: "Analyzing data... [Stage 1 runs]

Analyzed 2 datasets:

- `rice_production.csv`: 1200 rows, 8 columns, Quality: 92%
 - Has datetime column (Year)
 - Potential targets: Production, Yield
- `weather_data.csv`: 3650 rows, 6 columns, Quality: 88%
 - Has datetime column (Date)
 - Potential targets: Temperature, Rainfall

Generating task proposals... [Stage 2 runs]

Generated 4 task proposals:

- TSK-001: Forecast rice production for next 5 years
- TSK-002: Predict yield based on weather factors
- TSK-003: Analyze rainfall patterns
- TSK-004: Multi-year production trend analysis

”

User: "Run task 1"

Assistant: "Starting pipeline execution for TSK-001..."

Stages: 3 → 3B → 3.5A → 3.5B → 4 → 5

[Pipeline executes for 18 minutes]

Pipeline completed successfully!

Results:

- Method: XGBoost Regressor
- RMSE: 45.3
- MAE: 32.1
- R2: 0.87
- Visualization report created with 4 plots

"

User: "Show me the plots"

Assistant: "Visualization Report for TSK-001:"

1. Predictions vs. Actuals (scatter)

- Shows how predicted production values compare to ground truth
- Strong correlation (R2=0.87)
- Key insight: Model slightly underestimates high production years

2. Residual Analysis (histogram)

- Residuals approximately normally distributed
- Mean residual near zero (unbiased)
- Key insight: Few outliers, model is well-calibrated

3. Feature Importance (bar chart)

- Top features: Year, PrevYearProduction, AvgRainfall
- Key insight: Historical production is strongest predictor

4. Time Series Forecast (line plot)

- 5-year forecast: 2024-2028
- Predicted trend: Gradual increase
- Confidence intervals included

Plots saved in: output/stage5_out/"

8 Conclusions

This research demonstrates the design and implementation of a **truly agentic AI system** for conversational data analytics and forecasting. Our key contribution is showing that LLM-based systems can go beyond simple automation to exhibit intelligent decision-making, dynamic routing, and adaptive reasoning.

1. Multi-Agent Architecture with Intelligent Routing

The Conversation Agent serves as an intelligent router that *understands* user intent and *decides* where to dispatch requests:

- Data exploration queries → EDA Agent (autonomous code generation)
- Data analysis requests → Analysis Pipeline (Stages 1-2)

- Forecasting tasks → Forecasting Pipeline (Stages 3-6)

This is fundamentally different from automation—the system makes decisions based on understanding, not predefined rules.

2. Autonomous Agents with ReAct Reasoning

Each agent follows the ReAct paradigm: *think* → *act* → *observe* → *decide*. Agents don't just execute steps—they reason about their situation and adapt their approach.

3. EDA Agent: Code-Writing Intelligence

The EDA Agent is a key innovation—it doesn't use hardcoded analysis functions. Instead, it *writes custom Python code* for each user query, demonstrating true autonomous problem-solving.

4. Transparent Thought Process

Stage 6 provides complete transparency by documenting how each agent reasoned about its task. Users can understand *why* specific methods were selected, not just *what* was selected.

5. State-Based Multi-Agent Coordination

The system manages state across agent boundaries using LangGraph's StateGraph and MemorySaver checkpointing. This enables seamless handoffs between agents and resumption from any point.

What Makes This Agentic, Not Automated:

- **Decision-Making:** System decides what to do based on understanding user intent
- **Dynamic Routing:** Different pathways activated based on query type
- **Adaptive Reasoning:** Agents try alternative approaches when methods fail
- **Code Generation:** EDA Agent writes custom code, not predefined functions
- **Transparent Thinking:** Full reasoning traces available for inspection

Our work demonstrates that agentic AI systems require more than just LLM calls—they need structured multi-agent architectures, specialized tools, systematic reasoning frameworks, and intelligent routing mechanisms.

9 Extensions and Future Work

E1: Parallel Stage Execution

Currently, stages execute sequentially. When multiple independent tasks need processing (e.g., visualizing different aspects), parallel execution could reduce latency.

Implementation approach:

- Modify master orchestrator to spawn concurrent stage agents
- Use LangGraph's parallel edge support
- Aggregate results from parallel branches

E2: Expanded Method Library

Add support for:

- Deep learning (LSTM, GRU, Transformers)
- Ensemble methods (Stacking, Blending)
- Bayesian methods (PyMC, Stan)
- Custom user-defined methods via code upload

E3: Multi-Task Handling

Enable processing multiple tasks concurrently:

- "Compare forecasts for sales and revenue"
- Parallel execution of independent tasks

- Comparative visualization across tasks

E4: Interactive Refinement

Allow users to refine proposals:

- "Use XGBoost instead of Random Forest"
- "Add more lag features"
- "Focus on recent 2 years only"

E5: Causal Inference Integration

Extend from predictive to causal forecasting:

- Identify causal relationships in data
- Propose interventions ("What if we increase marketing spend?")
- Estimate treatment effects

Requires integration of causal discovery methods (PC algorithm, DAG learning) and causal inference frameworks (DoWhy, EconML).

9.1 Broader Impact

The conversational forecasting pipeline has potential applications across industries:

Scientific Research:

- Accelerate exploratory data analysis in biology, physics, climate science
- Enable non-statistician researchers to apply advanced methods
- Increase reproducibility through automated workflows

Business Intelligence:

- Empower business analysts to perform forecasting without data science teams
- Reduce time-to-insight for strategic decision making
- Enable SMBs to access advanced analytics

Education:

- Teaching tool for data science concepts through interactive exploration
- Students learn by asking questions and inspecting agent reasoning
- Democratize access to advanced analytics education

Public Policy:

- Enable government agencies to forecast economic indicators
- Support evidence-based policy making
- Increase transparency through explainable forecasts

References

- [1] Lei Wang, Chen Ma, Xueyang Feng, et al., *A Survey on Large Language Model based Autonomous Agents*, arXiv:2308.11432, August 2023 (updated March 2025). <https://arxiv.org/abs/2308.11432>
- [2] Shunyu Yao, Jeffrey Zhao, Dian Yu, et al., *ReAct: Synergizing Reasoning and Acting in Language Models*, ICLR 2023. <https://arxiv.org/abs/2210.03629>
- [3] Patara Trirat, Wonyong Jeong, Sung Ju Hwang, *AutoML-Agent: A Multi-Agent LLM Framework for Full-Pipeline AutoML*, arXiv:2410.02958, October 2024. <https://arxiv.org/abs/2410.02958>
- [4] Shaolin Wu et al., *Agent AI with LangGraph: A Modular Framework for Enhancing Machine Translation Using Large Language Models*, arXiv:2412.03801, December 2024. <https://arxiv.org/abs/2412.03801>
- [5] Hao Rao, Jinhui Yuan, Mingji Han, et al., *DataStates-LLM: Lazy Asynchronous Checkpointing for Large Language Models*, arXiv:2406.10707, June 2024. <https://arxiv.org/abs/2406.10707>
- [6] Arvind Narayan, Sherry Cheng, et al., *Conversational Interfaces for Data Analytics: A Survey*, ACM Computing Surveys, 2023.
- [7] Harrison Chase et al., *LangChain: Building applications with LLMs through composability*, 2022. <https://github.com/langchain-ai/langchain>
- [8] LangChain AI, *LangGraph: Build stateful, multi-actor applications with LLMs*, 2023. <https://github.com/langchain-ai/langgraph>
- [9] Qwen Team, *Qwen2.5: Large Language Models for Diverse Applications*, 2024. <https://github.com/QwenLM/Qwen>
- [10] Samuel Colvin et al., *Pydantic: Data validation using Python type hints*, 2023. <https://docs.pydantic.dev/>